

任务 7: MyBatis-Plus 多表联查与 Redis 缓存集成

在真实的生产环境中，高频访问的数据（如用户信息）如果每次都直接查询数据库，会给数据库带来巨大的压力。本任务将通过 **Redis** 作为缓存层，配合 **MyBatis-Plus** 自定义多表联查，实现高性能的用户详情管理。

1. 环境准备：集成 Redis

首先，确保你的 `pom.xml` 中引入了 Redis 的起步依赖：

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>

<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>cn.hutool</groupId>
  <artifactId>hutool-all</artifactId>
  <version>5.8.25</version>
</dependency>
```

在 `application.yml` 中配置 Redis 连接：

YAML

```
spring:
  data:
    redis:
      host: 127.0.0.1
```

```
port: 6379
database: 0
```

- `host` : Redis 服务地址
- `port` : Redis 默认端口是 `6379`
- `database` : 默认使用第 0 个逻辑库

同时确保本地 Redis 已启动, 可以使用 `redis-cli ping` 命令行查看redis启动情况。

2. 定义视图对象 (VO) 与 Mapper 联查

由于本任务是**两张表联查后的结果**, 因此不建议直接返回单表 Entity。

为了承接 `sys_user` 和 `user_info` 两张表合并后的结果, 我们需要创建一个 VO (View Object)。

并确保在系统Entity和数据库中存在两张表: 用户基础表 `sys_user`、用户扩展信息表 `user_info`。

2.1 创建 UserDetailVO

在 `vo` 包下创建 `UserDetailVO`, 包含核心账号信息和详细个人信息:

Java

```
import lombok.Data;

@Data
public class UserDetailVO {
    private Long userId;
    private String username;
    private String realName;
    private String phone;
    private String address;
}
```

- 这里使用 `@Data` 自动生成 getter / setter
- 当前任务采用的是 `StringRedisTemplate` + JSON 字符串缓存, 因此这里**不强制要求实现** `Serializable`

- 该 VO 的职责是：**承接多表联查结果，并作为接口返回对象**，后期直接返回给前端。

2.2 创建 UserInfoMapper 自定义 SQL

在 `mapper` 包下创建 `UserInfoMapper`，手写 `LEFT JOIN` 语句：

Java

```
import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import org.apache.ibatis.annotations.Mapper;
import org.apache.ibatis.annotations.Param;
import org.apache.ibatis.annotations.Select;

@Mapper
public interface UserInfoMapper extends BaseMapper<UserInfo> {

    @Select("""
        SELECT
            u.id AS userId,
            u.username,
            i.real_name AS realName,
            i.phone,
            i.address
        FROM sys_user u
        LEFT JOIN user_info i ON u.id = i.user_id
        WHERE u.id = #{userId}
        """)
    UserDetailVO getUserDetail(@Param("userId") Long userId);
}
```

`BaseMapper` 和 `Wrapper` 更适合**单表操作**，因此在数据量不大时，复杂多表联查优先考虑使用自定义 Mapper 方法 + 原生 SQL/XML。

3. 实现核心业务逻辑 (含缓存控制)

我们将采用 **Cache-Aside（旁路缓存）** 策略。

查询：先查缓存，查不到再查 DB 并回写缓存。

更新/删除：先操作 DB，成功后删除旧缓存（保证一致性）。

3.1 修改 UserServiceImpl

重构业务层代码，注入 `StringRedisTemplate`，并根据逻辑写缓存查、改、删：

Java

```
@Override
public Result<UserDetailVO> getUserDetail(Long userId) {
    String key = CACHE_KEY_PREFIX + userId;

    // 1. 先查缓存
    String json = redisTemplate.opsForValue().get(key);
    if (json != null && !json.isBlank()) {
        try {
            UserDetailVO cacheVO = JSONUtil.toBean(json, UserDetailVO.class);
            return Result.success(cacheVO);
        } catch (Exception e) {
            // 缓存数据异常，删掉脏缓存，继续查数据库
            redisTemplate.delete(key);
        }
    }

    // 2. 查数据库
    UserDetailVO detail = userInfoMapper.getUserDetail(userId);
    if (detail == null) {
        return Result.error(ResultCode.USER_NOT_EXIST);
    }

    // 3. 写缓存
    redisTemplate.opsForValue().set(
        key,
        JSONUtil.toJsonStr(detail),
        10,
        TimeUnit.MINUTES
    );

    return Result.success(detail);
}

@Override
@Transactional
```

```
public Result<String> updateUserInfo(UserInfo userInfo) {  
    // 参数校验,userInfo 不能为空,并且 userId 不能为空,后面删除 Redis 缓存时  
    if (userInfo == null || userInfo.getUserId() == null) {
```

4. 添加Controller层代码

在controller层添加如下接口代码。

Java

```
// 5. 查询用户详情（多表联查 + Redis）  
@GetMapping("/{id}/detail")  
public Result<UserDetailVO> getUserDetail(@PathVariable("id") Long userId)  
    return userService.getUserDetail(userId);  
}  
  
// 6. 更新用户扩展信息  
@PutMapping("/{id}/detail")  
public Result<String> updateUserInfo(@PathVariable("id") Long userId,  
    @RequestBody UserInfo userInfo) {  
    userInfo.setUserId(userId);  
    return userService.updateUserInfo(userInfo);  
}  
  
// 7. 删除用户  
@DeleteMapping("/{id}")  
public Result<String> deleteUser(@PathVariable("id") Long userId) {  
    return userService.deleteUser(userId);  
}
```

附加题 1: Redis 联动实战验证

重启项目，确保你的本地 Redis 服务已启动。

1. 测试查询并观察缓存:

- 调用 `GET http://localhost:8080/api/users/1/detail`。
- 查看控制台日志，确认触发了 SQL 查询。
- 打开 Redis 客户端（`redis-cli`）输入 `keys user:detail:*`，确认出现了

`user:detail:1` 这个键。

- 再次调用接口，观察控制台此时不应有 SQL 输出（说明命中缓存）。

2. 测试数据更新的一致性:

- 使用 `PUT` 请求更新用户个人信息。
- 检查 Redis，确认对应的键已被自动删除。
- 再次查询详情，观察 SQL 重新触发并加载了修改后的姓名。

提交要求

1. 测试与任务成功示例:

- 任务成功 1：Redis 客户端（如 Another Redis Desktop Manager）中显示缓存 Key 的快照。
- 任务成功 2：更新数据后，Redis 缓存被成功删除的证据。

2. 文档提交: 将 `UserInfoMapper`、`UserDetailVO` 以及 Service、Controller 层改造后的核心代码推送至个人仓库，并将截图粘贴至 Word 文档按时提交。